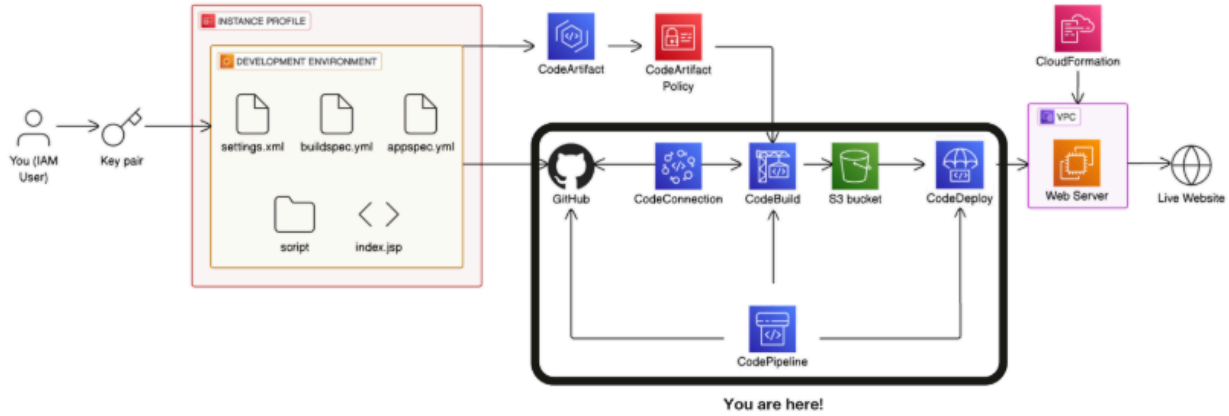
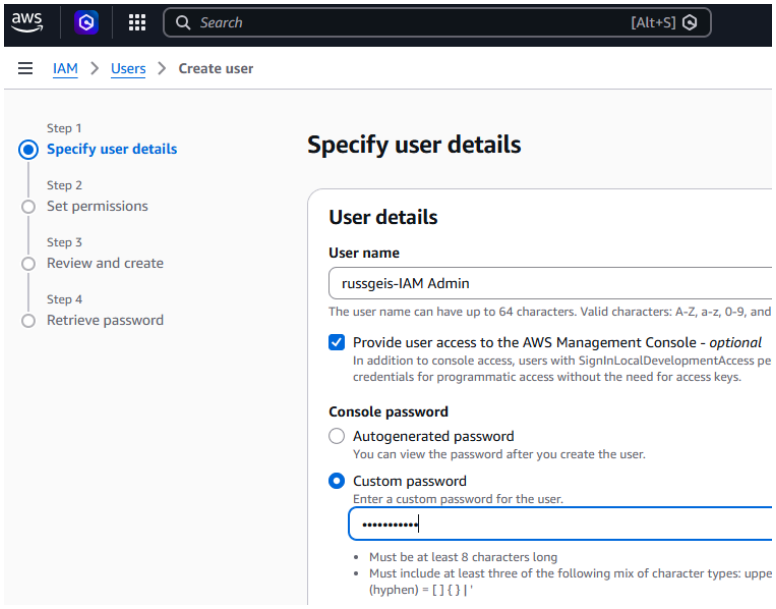
AWS CI/CD Pipeline

In this lab, I will build a CI/CD Pipeline in AWS. CI/CD pipeline automatically builds, tests, and deploys your code changes. Instead of manually running these steps, CodePipeline manages the entire process from code commit to deployment.



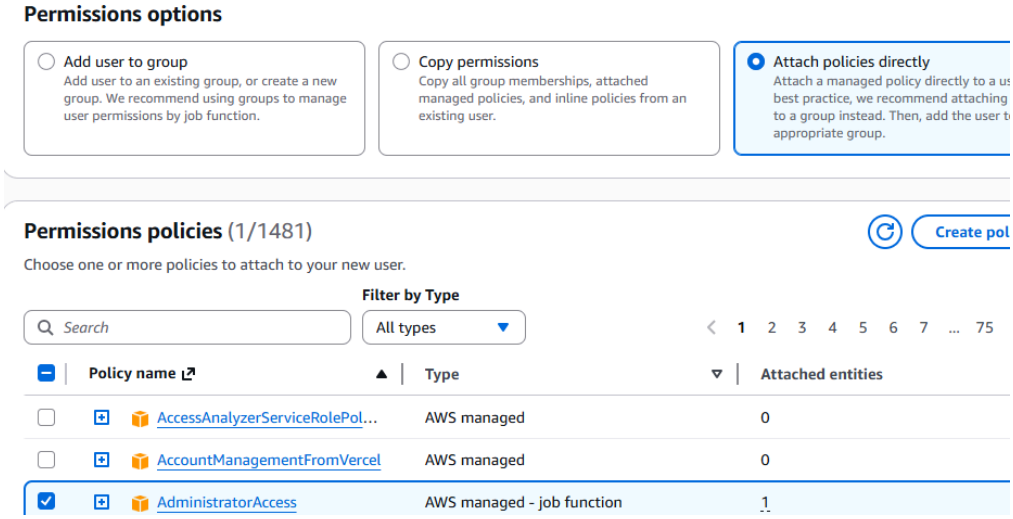
AWS Web App with AWS

1. Log into your AWS account as a root user
2. Go to the IAM console
3. Click Users
4. Create User
5. Make sure to click **Provide user access to the AWS Management Console - optional.**
6. Add custom password
7. Deselect **Users must create new password at next login**

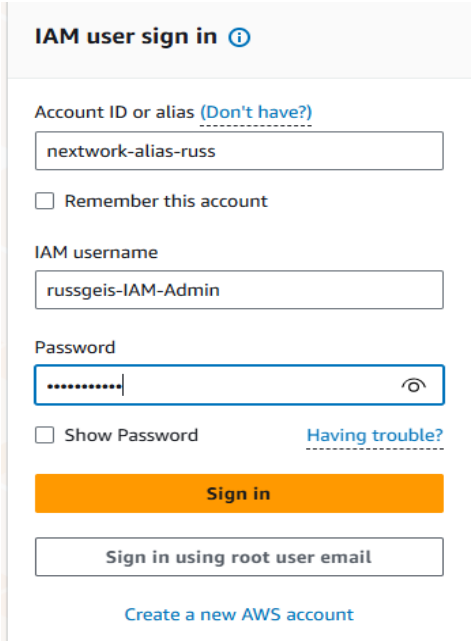


8. Select **Attach Policies Directly**

9. Add **Administrator Access**



10. Select Next
11. Choose **Create user.**
12. Voilà - you've just created your new user! **Stay on this page**.
13. Choose **Download .csv file.**
14. Copy the Console sign in user URL
15. Now you're ready to start using your IAM user. 🏁
16. Log out of your root user's AWS Account.
17. Paste and go to your copied console sign-in URL.
18. Open your downloaded .csv file containing your user's access instructions.
19. Log in using your IAM user's username and password in the .csv file.



20. Once you log in, verify your region is one closest to you

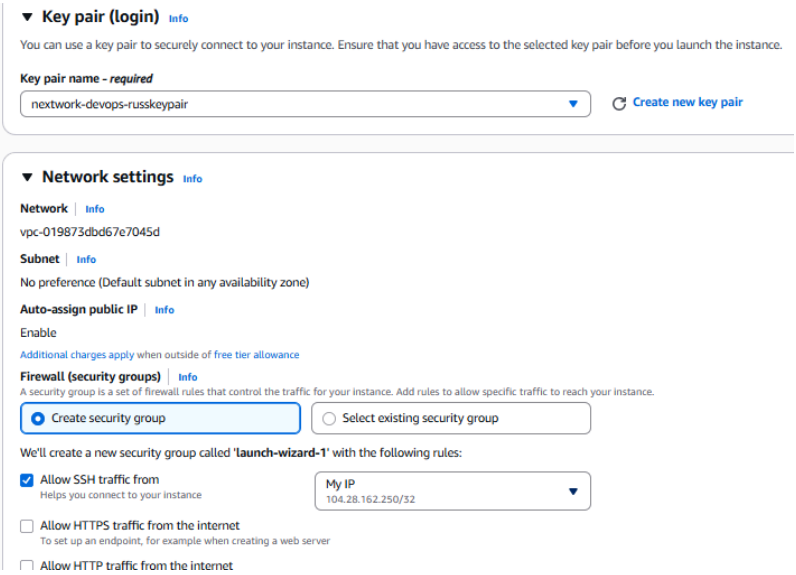
Launching EC2 instance

- In your EC2 console, select **Instances** from the left hand navigation panel.
- Choose **Launch instances**.
- Name your instance
- Choose **Amazon Linux 2023 kernel-6.1 AMI**
- Leave **t3.micro** (or **t2.micro** in some regions) under Instance type. Both are free tier eligible.
- Under **Key pair (login)**, choose **Create a new key pair.**
- Use nextwork-keypair as your key pair's name.
- Keep the **Key pair type** as **RSA**, and the **Private key file format** as **.pem**
- Select **Create key pair.**
- A new file will automatically download to your local computer - nice! This is your private key.
- Before we lose track of our .pem file, let's organise it in our computer.
- Head to your local computer's desktop.
- Create a new folder in your desktop called **DevOps**.
- Move your .pem file from your **Downloads** folder into your **DevOps** folder.
- Note the path of your .pem file:
- Right-click on nextwork-keypair.pem in your DevOps folder, hold **Option**, and select **Copy "nextwork-keypair.pem" as Pathname**.

Your path will look like:

/Users/maximus/Desktop/DevOps/nextwork-keypair.pem

- Save this file path in notepad
- Back to our EC2 instance setup, head to the **Network settings** section.
- For **Allow SSH traffic from**, select the dropdown and choose **My IP**. This makes sure only you can access your EC2 instance.
- Double check that the IP address under **My IP** is correct - you can check your IP by clicking here.
- If your IP address is different from what's under **My IP**, select **Custom** from the dropdown instead. Enter your IP and make sure to add a /32 to the end e.g. 012.345.678.9/32
- Leave the default values for the remaining sections.



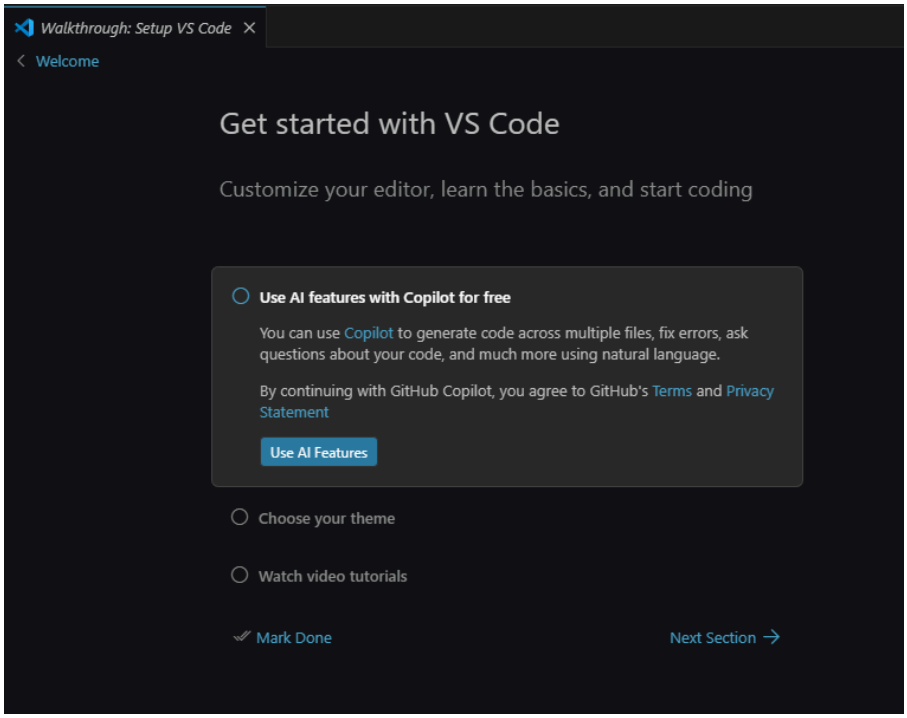
Install VS Code

Now that your EC2 instance is up and running, how are we going to get inside your instance and set up a web app?

We'll use Visual Studio Code to connect with your instance, so you can create and edit your web app's code.

First things first, let's install VS Code.

- Head to the [Visual Studio Code website](#).
- Install VS Code by following the installation instructions for your OS (Mac, Windows, or Linux). The download page will recommend the right version for your computer automatically.
- On the **Select Additional Tasks** screen, leave the default options and click **Next**.



-
- Select **Terminal** from the top menu bar.
- Select **New Terminal** from the dropdown.
- Navigate your terminal to the DevOps folder. You'll do this by entering this command in the terminal: **cd ~\Desktop\DevOps**
- Run **dir** to confirm your .pem file is in this folder.
- First, find your Windows username by running this in your terminal: whoami
- The output will look something like pcr\administrator. The part **before** the backslash (pcr) is your PC name. The part **after** the backslash (administrator) is your **username**. You only need the username.
- Now run these three commands **one at a time**: **icacls "nextwork-keypair.pem" /reset**
- icacls "nextwork-keypair.pem" /grant:r "Enter your Windows username:R"
- icacls "nextwork-keypair.pem" /inheritance:r

```
PS C:\Users\russe\OneDrive\Desktop\DevOps> dir
la---           4/9/2026  2:18 PM          1678 nextwork-devops-russkeypair.pem

PS C:\Users\russe\OneDrive\Desktop\DevOps> whoami
desktop7090\russe
PS C:\Users\russe\OneDrive\Desktop\DevOps> icacls "nextwork-keypair.pem" /reset
>>
processed file: nextwork-keypair.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\russe\OneDrive\Desktop\DevOps> icacls "nextwork-keypair.pem" /grant:r "desktop7090:R"
>>
processed file: nextwork-keypair.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\russe\OneDrive\Desktop\DevOps> icacls "nextwork-keypair.pem" /inheritance:r
>>
processed file: nextwork-keypair.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\russe\OneDrive\Desktop\DevOps>
```

These commands lock down your .pem file so only you can read it (which SSH requires):

- /reset removes all existing permissions so you start with a clean slate.
- /grant:r "YOURUSERNAME:R" gives **only your user account** read access to the file.
- /inheritance:r stops the file from inheriting permissions from its parent folder, keeping it locked down.

AWS EC2 Instance

- Head back to your AWS Management Console.
- Click on **Instances** in the breadcrumb at the top of the page.
- Click on the checkbox next to your EC2 instance to view its details.
- Under the **Details** tab, look for **Public DNS** (also called Public IPv4 DNS). You'll need this in the next instruction!
- Copy down you Public DNS in notepad
- Now back in VS Code. Open terminal and paste this code: **ssh -i C:\Users\russe\OneDrive\Desktop\DevOps\nextwork-devops-keypair.pem** ec2-3-144-220-191.us-east-2.compute.amazonaws.com
-
- Your terminal will ask if you want to continue connecting to this EC2 instance. This is SSH's way of asking if you trust this server.
- Enter yes to continue connecting.
- (HINT: if this is an existing instance, you may need to refresh the security group to target “My IP” again)

```
PS C:\Users\russe\OneDrive\Desktop\DevOps> icacls "nextwork-devops.pem" /reset
>> icacls "nextwork-devops.pem" /grant:r "russe:R"
>> icacls "nextwork-devops.pem" /inheritance:r
>>
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\russe\OneDrive\Desktop\DevOps> ssh -i C:\Users\russe\OneDrive\Desktop\DevOps\nextwork-devops.pem ec2-user@ec2-3-128-186-96.us-east-2.c
aws.com
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Apr 20 20:26:37 2026 from 104.28.152.216
[ec2-user@ip-172-31-18-246 ~]$
```

YOU ARE CONNECTED

Installing Apache Movan and Corretto 8

Install Apache Maven by copying and pasting all of these commands into your terminal:

wget https://archive.apache.org/dist/maven/maven-3/3.5.2/binaries/apache-maven-3.5.2-bin.tar.gz

sudo tar -xzf apache-maven-3.5.2-bin.tar.gz -C /opt

echo "export PATH=/opt/apache-maven-3.5.2/bin:$PATH" >> ~/.bashrc

source ~/.bashrc

- Now we're going to install Java 8, or more specifically, Amazon Corretto 8.
- Install Java and set up its location by pasting these commands:

sudo dnf install -y java-1.8.0-amazon-corretto-devel

export JAVA_HOME=/usr/lib/jvm/java-1.8.0-amazon-corretto.x86_64

export PATH=/usr/lib/jvm/java-1.8.0-amazon-corretto.x86_64/jre/bin/:$PATH

1. To verify that Maven is installed correctly, run the following command next:
- mvn -v
2. To verify that you've installed Java 8 correctly, run this next:
- java -version

Creating the web app

Use **mvn** to generate a Java web app. To do this, use these commands:

mvn archetype:generate -DgroupId=com.nextwork.app -DartifactId=nextwork-web-project -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false

```
maven-archetype-webapp-1.0.jar (3.9 kB at 141 kB/s)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/ec2-user
[INFO] Parameter: package, Value: com.nextwork.app
[INFO] Parameter: groupId, Value: com.nextwork.app
[INFO] Parameter: artifactId, Value: nextwork-web-project
[INFO] Parameter: packageName, Value: com.nextwork.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ec2-user/nextwork-web-project
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 15.466 s
[INFO] Finished at: 2026-04-09T21:39:41Z
[INFO] Final Memory: 18M/83M
[INFO] ------------------------------------------------------------------------
[ec2-user@ip-172-31-25-188 ~]$
```

Adding the VS Code plugin

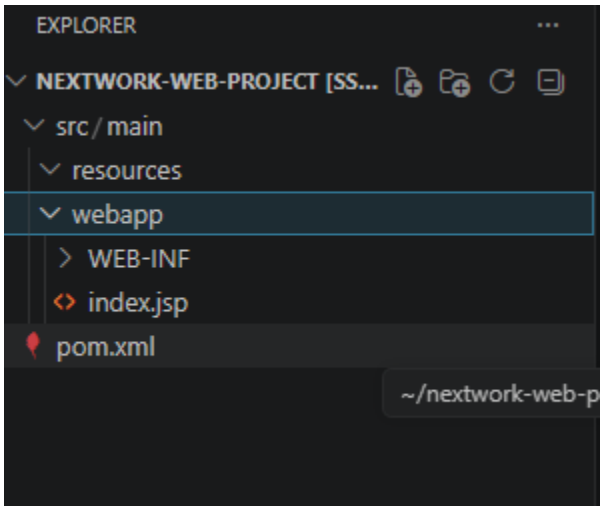
- Click on the Extensions icon at the side of your VS Code window.
- In the search bar, type Remote SSH and click **Install** for the extension.
- Click on the double arrow icon at the bottom left corner of your VS Code window. This button is a shortcut to use Remote - SSH.
- Select **Connect to Host...**
- Select **+ Add New SSH Host...**
- A text box will appear at the top of your VS Code window. This is where you type the SSH command. Enter the **same SSH command** you used to connect in Step 4:
    - ssh -i your .pem file path ec2-user@your Public DNS
    - Mine is this: **ssh -i "C:\Users\russe\OneDrive\Desktop\Devops\nextwork-devops-russkeypair.pem" ec2-user@ec2-3-128-190-89.us-east-2.compute.amazonaws.com**
    - Press **Enter.**
    - VS Code will ask you to select a configuration file**. Select C:\Users\\.ssh\config (this should be the first option**
    - A Host added! popup will confirm that you've set up your SSH Host - yay

    - Select the blue **Open Config button** on that popup.
    - Confirm that all the details in your configuration file look correct:
        - Host and HostName should both match your EC2 instance's Public DNS.
        - IdentityFile should be the path to your **nextwork-keypair.pem** file on your local computer.
        - User should say **ec2-user**

- Now you're ready to connect VS Code with your EC2 instance!
- Click on the double arrow button on the bottom left corner and select **Connect to Host** again.
- You should now see your EC2 instance listed at the top.
- Select the EC2 instance and off we goooooooooo to a new VS Code window ✈️
- If VS Code asks you to select the platform of the remote host, choose **Linux.** This isn't asking about your computer - it's asking about the machine you're connecting to. Your EC2 instance runs **Amazon Linux,** so select **Linux** even if you're on Windows or Mac.
- Check the bottom right hand corner of your new VS Code window - it should show your EC2 instance's Public DNS.

Now let's open up your web app's files.

- From VS Code's left hand navigation bar, select the **Explorer icon**
  - Select **Open folder.**
  - At the top of your VS Code window, you should see a drop down of different file and folder names. Ooooo, this is VS Code asking you which specific file/folder you'd like to open!
  - Enter **/home/ec2-user/nextwork-web-project.**
  - Press **OK.**
  - VS Code might show you a popup asking if you trust the authors of the files in this folder. If you see this popup, select **Trust Folder & Continue.**
  - Check your VS Code window's file explorer again - a folder called **nextwork-web-project** is here!



  - From your file explorer, click into **index.jsp.**
  - Let's try modifying index.jsp by changing the placeholder code to the code snippet below. Don't forget to enter your name in the following code

- **<html> <body> <h2>Hello !</h2> This is my NextWork web application working! </body> </html>**
- Save the changes you've made to index.jsp by selecting Command/Ctrl + S on your keyboard.
- **We have created a web app using VS Code, Maven and Corretto 8.**

Install GitHub

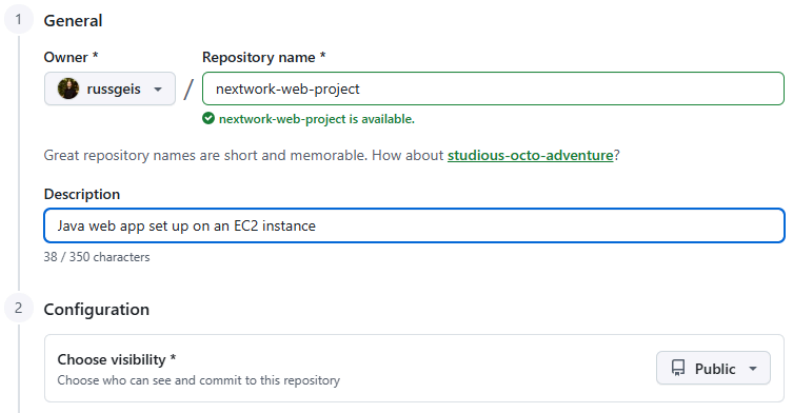
Github helps connect you with you source code repository

1. Use this script in VS Code terminal to add GitHub

   sudo dnf update -y

   sudo dnf install git -y

2. **Go to GitHub and log into your account**
3. Select **Create repository.**
4. This loads up a new page where you can create a repository.
5. Under **Owner,** click on the **Choose an owner** dropdown and select your GitHub username.
6. Under **Repository name, enter nextwork-web-project**
7. For the Description, enter **Java web app set up on an EC2 instance.**
8. Choose your visibility settings. We'd recommend selecting **Public** to make your repository available for the world to see.



In the blue section of the page titled **Quick setup - if you've done this kind of thing before,** copy the HTTPS URL to your repository page. It will look like https://github.com/username/nextwork-web-project.git

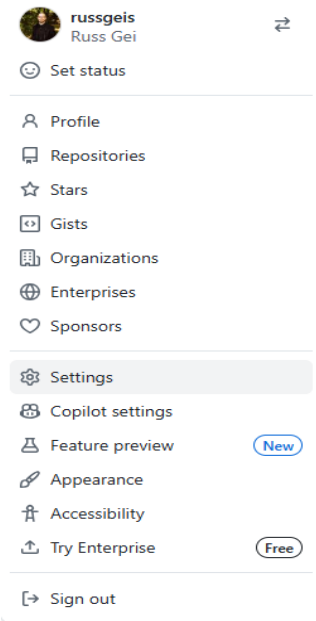
Back in VSCode type:

Git remote add origin (add URL you just copied)

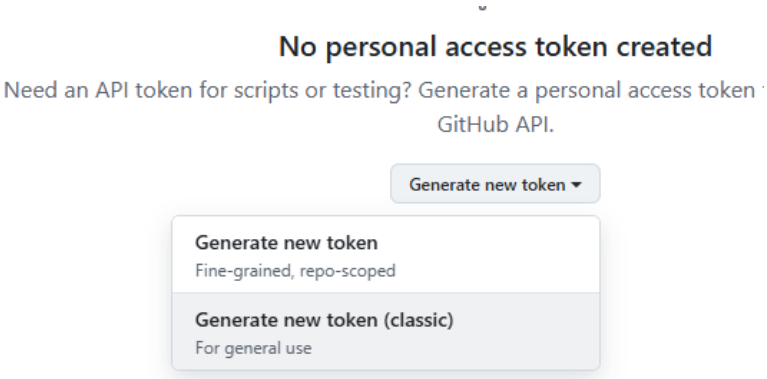
```
Initialized empty Git repository in /home/ec2-user/.git/
[ec2-user@ip-172-31-26-93 ~]$ git remote add origin https://github.com/russgeis/nextwork-web-project.git
```

Add GitHub Token

- Head back into your browser with GitHub open.
- Select your profile icon and select **Settings.**

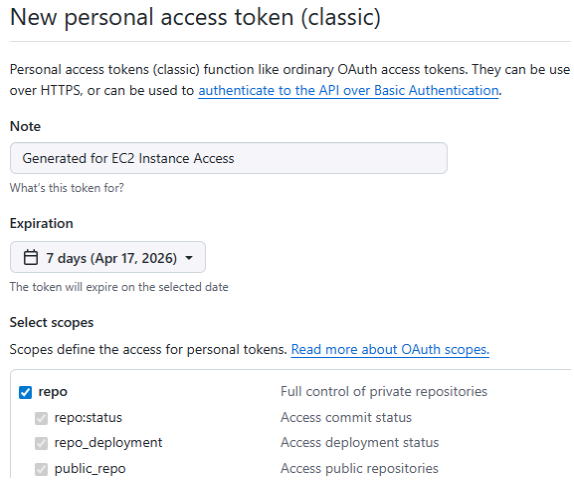


- Select **Developer settings** at the very bottom of the left hand navigation panel**.**
- Select **Personal access tokens.**
- Select **Tokens (classic).**
- Select **Generate new token.**
- Select **Generate new token (classic).**



- Give your token a descriptive note, like **Generated for EC2 Instance Access.**
- Lower the token expiration limit from **30 days to 7 days**

- Select the checkbox next to **repo**
- **Generate Token**

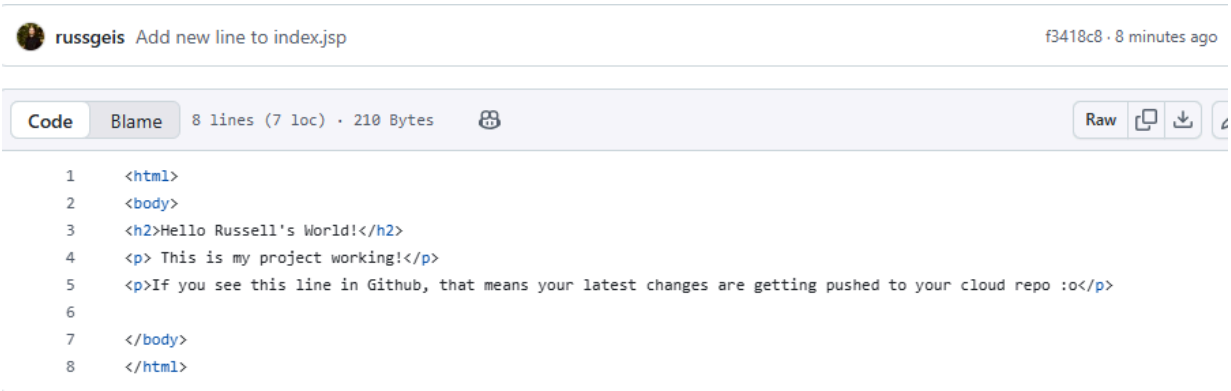


- Make sure to copy your token now. Keep it safe somewhere else, you won't be able to see your token once you close this tab
- Head to your VS Code terminal.
- Run **git push -u origin master** again, which will trigger Git to ask for your GitHub username.
- Enter your username again.
- ✋ **PAUSE**
- When Git asks for your password, **paste in your token instead**
- Head to your GitHub repository in your web browser.
- Refresh the page, and you'll see your web app files in the repository, along with the commit message you wrote.
- Run **git log** to see your history of commits, which also mentions the commit author's name.
- Run these commands in your terminal to manually set your name and email. Don't forget to replace with your name (keep the quote marks), and your email address.

```
[ec2-user@ip-172-31-18-246 nextwork-web-project]$ git config --global user.name "russgeis"
git config --global user.email russellsgeisler@gmail.com
[ec2-user@ip-172-31-18-246 nextwork-web-project]$ git add .
```

-

Your Second Commit

- Keep your GitHub page open, and switch back to your EC2 instance's VSCode window.
- Find **index.jsp** in your file navigator on the left hand panel.

- Find the line that says **This is my NextWork web application working!** and add this line below:
    - If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o
- In the terminal, let's stage our changes: **git add .**
- Ready to see what changes are staged? **Run git diff --staged**
- Nice, these changes are what we want to save and send to GitHub, lets do that with these commands:
    - git commit -m "Add new line to index.jsp"
    - git push
- You might need to enter your username and token again to complete your push.
- Head back to your GitHub tab and refresh it - do you see your changes now?